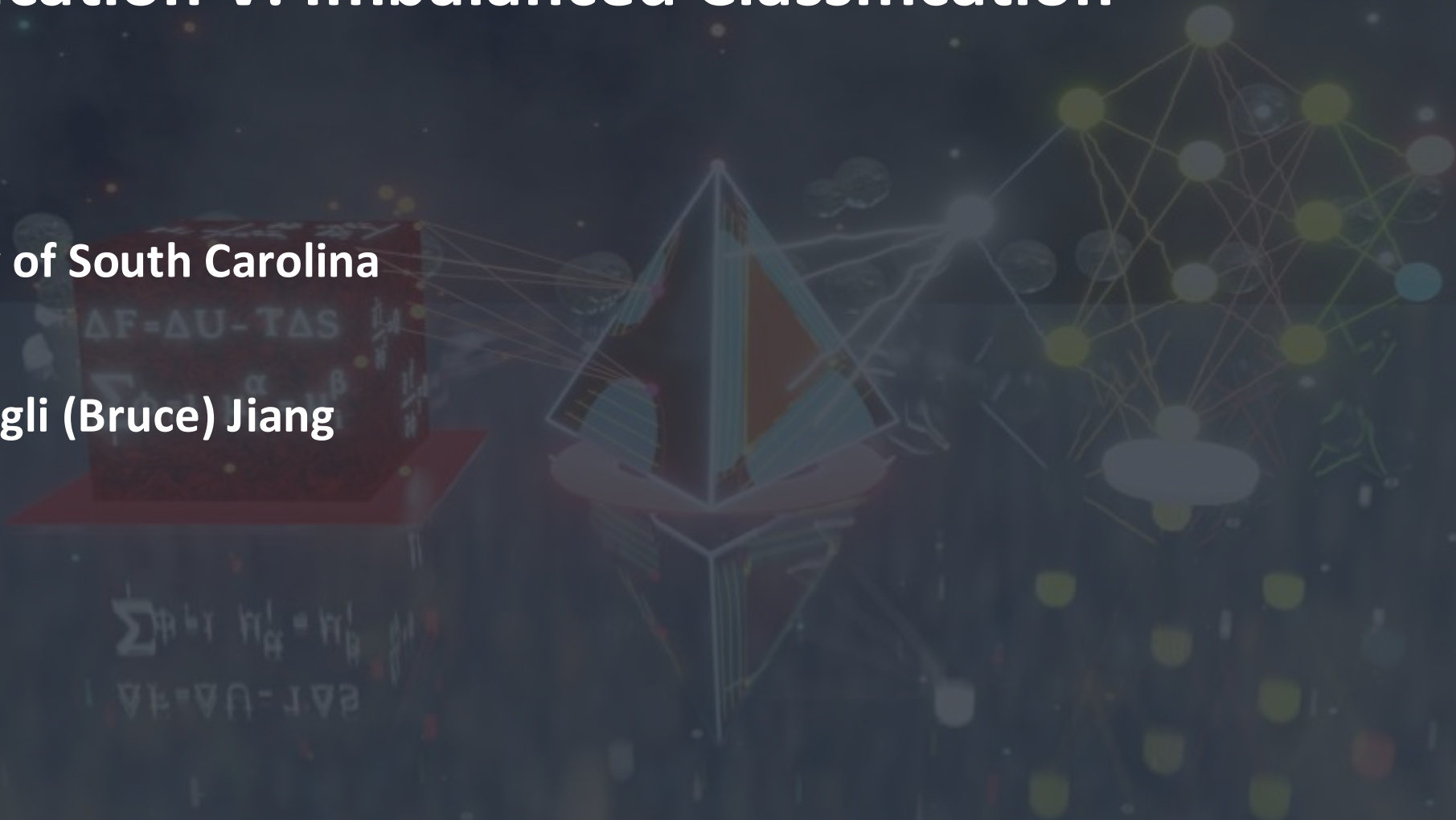


Classification V: Imbalanced Classification

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



From Lecture 15

Classification: Decision Trees and Random Forest

- *Trees select splits by reducing impurity.*
- *Random forests average predictions from different trees.*
- *Less-correlated tree errors can cancel through averaging.*

In this lecture, you will learn

- *Why accuracy can fail on imbalanced data.*
- *How to interpret ROC and precision–recall curves.*
- *How to Use thresholds and class weighting without data leakage.*

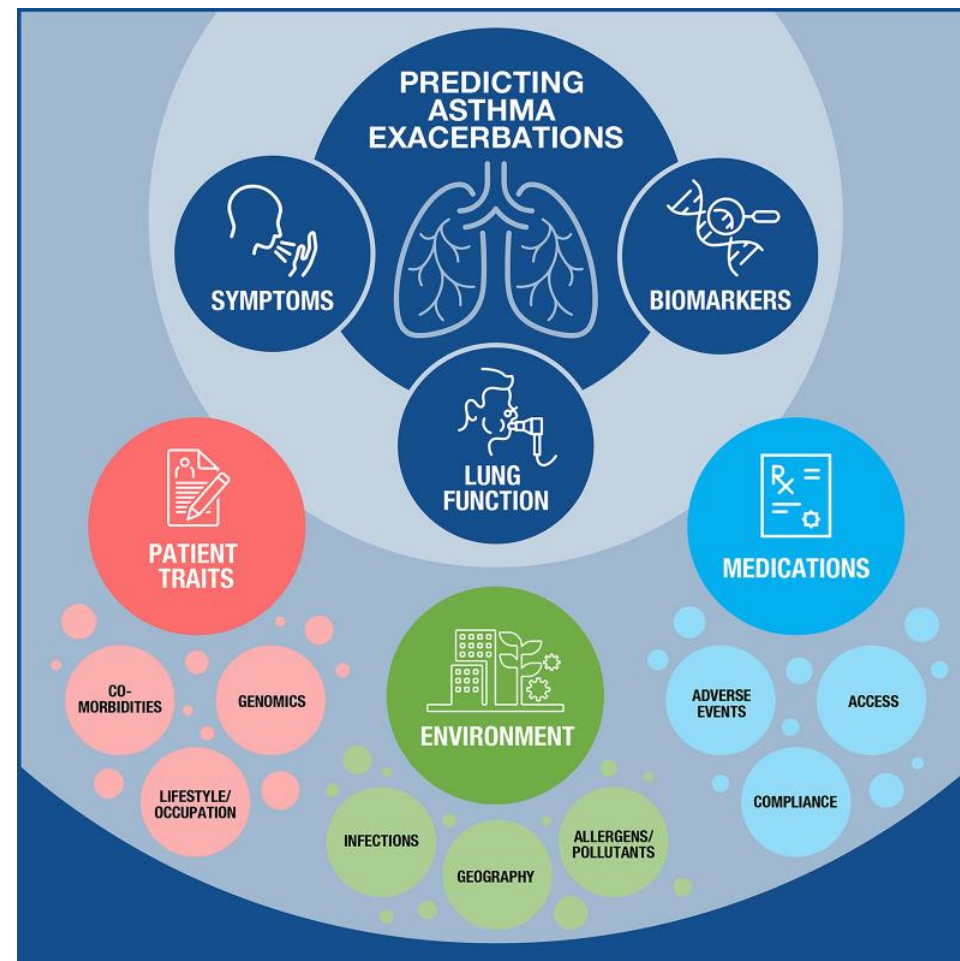
Why Rare Cases Matter

In many real classification problems, the event we care about is rare.

Examples:

- Disease screening → positive cases are uncommon
- Chemical/process fault detection → failures are rare
- Fraud detection → fraudulent transactions are rare
- Material failure → failure events are rare

The minority class may be rare, but it can be the most important class to detect.



Accuracy Can Hide Missed Positives

Suppose we have 1000 samples, with 990 negative, and 10 positive.

Now use the naive classifier:

Predict every sample as negative.

$$\text{accuracy} = \frac{990}{1000} = 0.99$$

But for the positive class:

$$TP = 0, FN = 10$$

Recall measures how many actual positive samples are detected.

$$\text{recall} = \frac{TP}{TP + FN} = 0$$

In an imbalanced problem, accuracy can be dominated by the majority class.

Precision: How Reliable Are Positive Predictions?

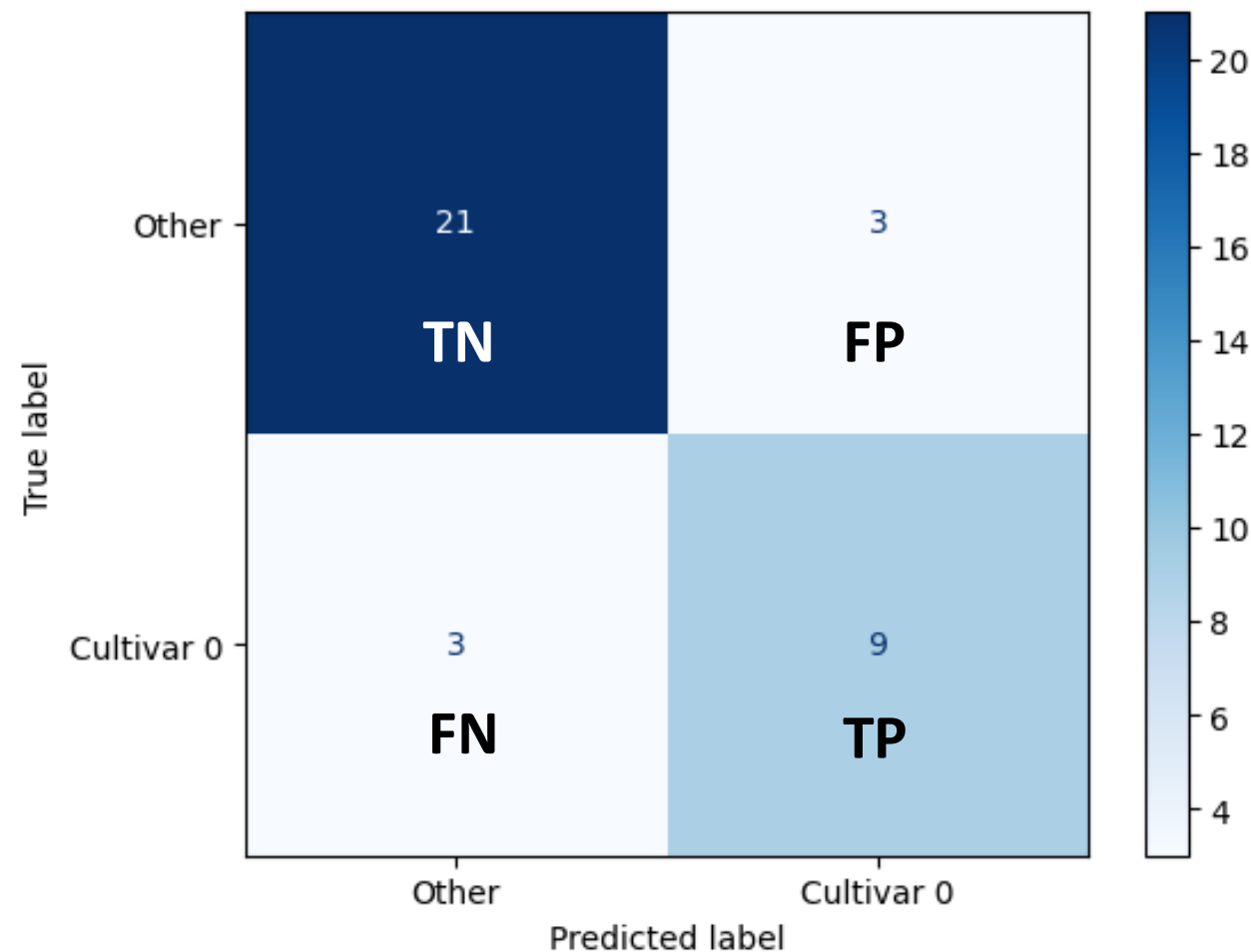
Precision measures how often a positive prediction is correct:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Use precision when **false positives are costly**. Here, it answers:

Of all samples predicted as Cultivar 0, what fraction actually belong to Cultivar 0?

Assume **Cultivar 0** is the positive class.



Recall: How Many Positives Did We Find?

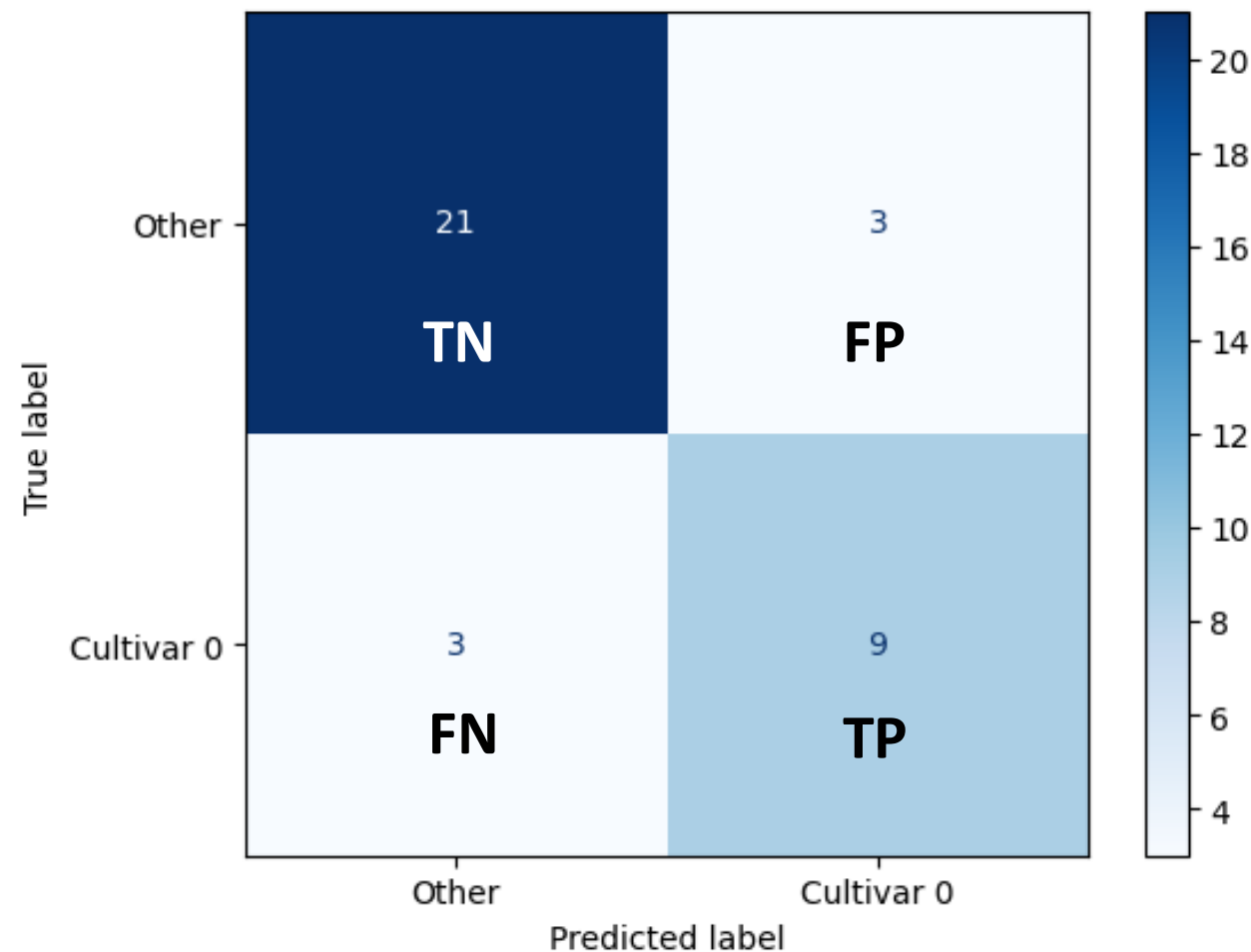
Recall measures how many actual positive samples are detected:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Use recall when **missing a positive sample is costly**. It answers:

Of all actual Cultivar 0 samples, what fraction did the model identify?

Assume **Cultivar 0** is the positive class.



Specificity: How Many Negatives Did We Reject?

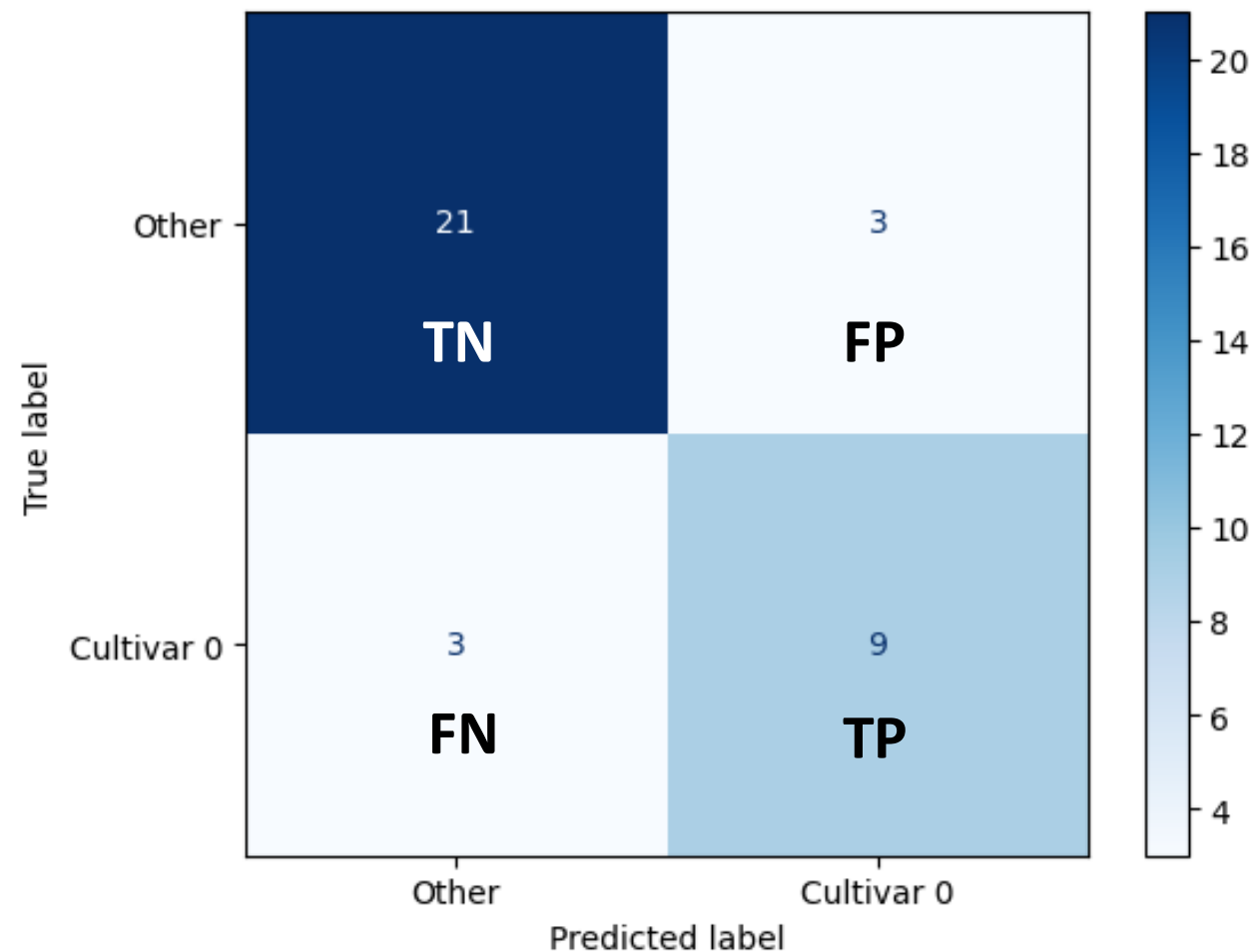
Specificity measures how well the model recognizes the negative class:

$$\text{Specificity} = \frac{TN}{TN + FP}$$

Use specificity when correctly rejecting negative samples matters. It answers:

Of all samples that are not Cultivar 0, what fraction did the model correctly reject?

Assume **Cultivar 0** is the positive class.



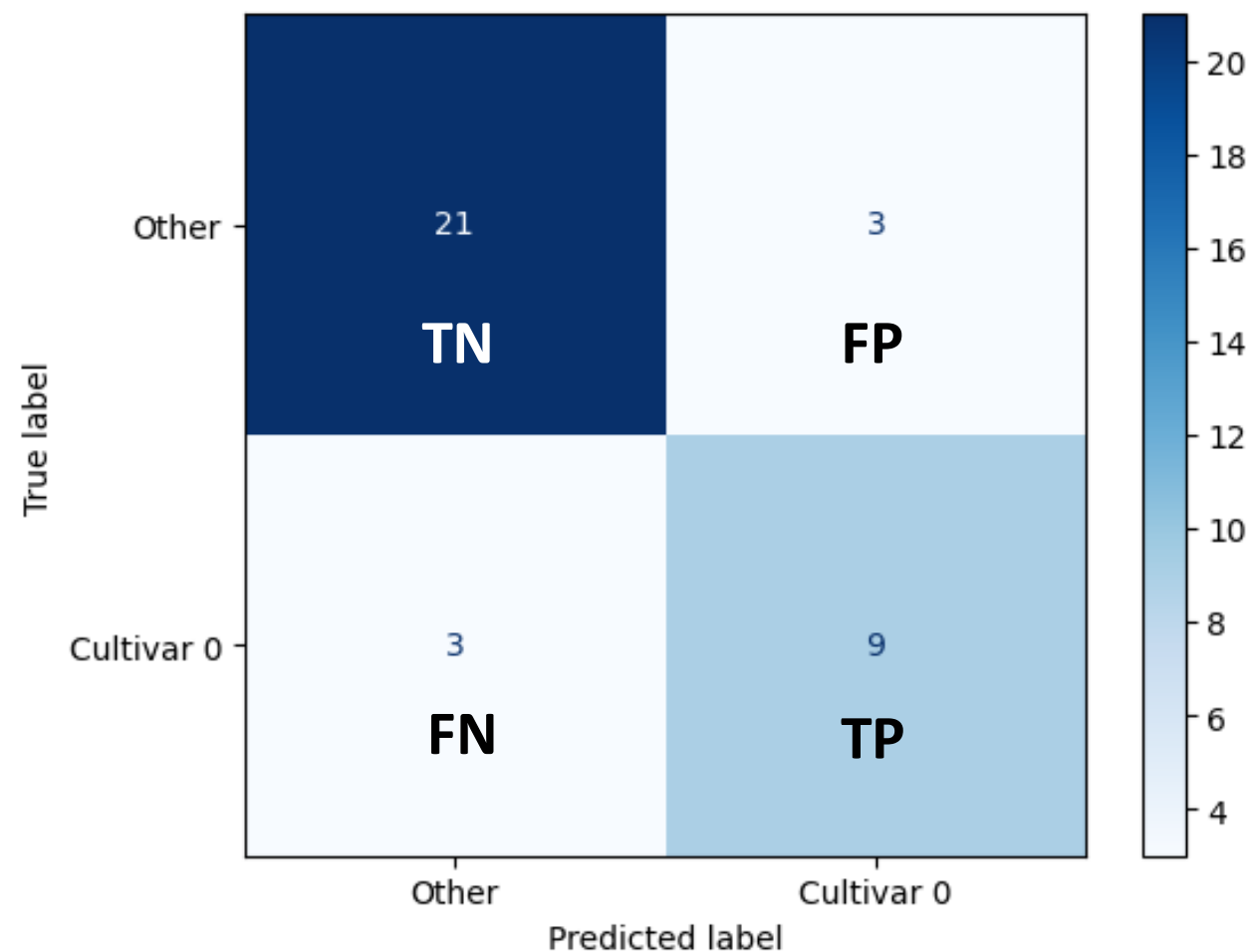
F1 Score: Combining Precision and Recall

The **F_1 score** is the harmonic mean of precision and recall:

$$F_1 = 2 \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
$$= \frac{2TP}{2TP + FP + FN}$$

Use F_1 when you want one score that balances **false positives** and **false negatives**, particularly when the classes are imbalanced.

Assume **Cultivar 0** is the positive class.



Turning Probabilities into Class Labels

$$\hat{y} = \begin{cases} 1, & P(y = 1 \mid \mathbf{x}) \geq t \\ 0, & P(y = 1 \mid \mathbf{x}) < t \end{cases}$$

- Logistic regression produces a score or probability.
- The threshold produces the class prediction.
- $t = 0.5$ is only a default, not necessarily the best threshold.

Sample	$P(y = 1)$
A	0.90
B	0.65
C	0.48
D	0.30

- At $t=0.5$, only A and B are predicted as positive.
- At $t=0.3$, A, B, C, D are all predicted as positive.

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{Precision} = \frac{TP}{TP + FP}$$

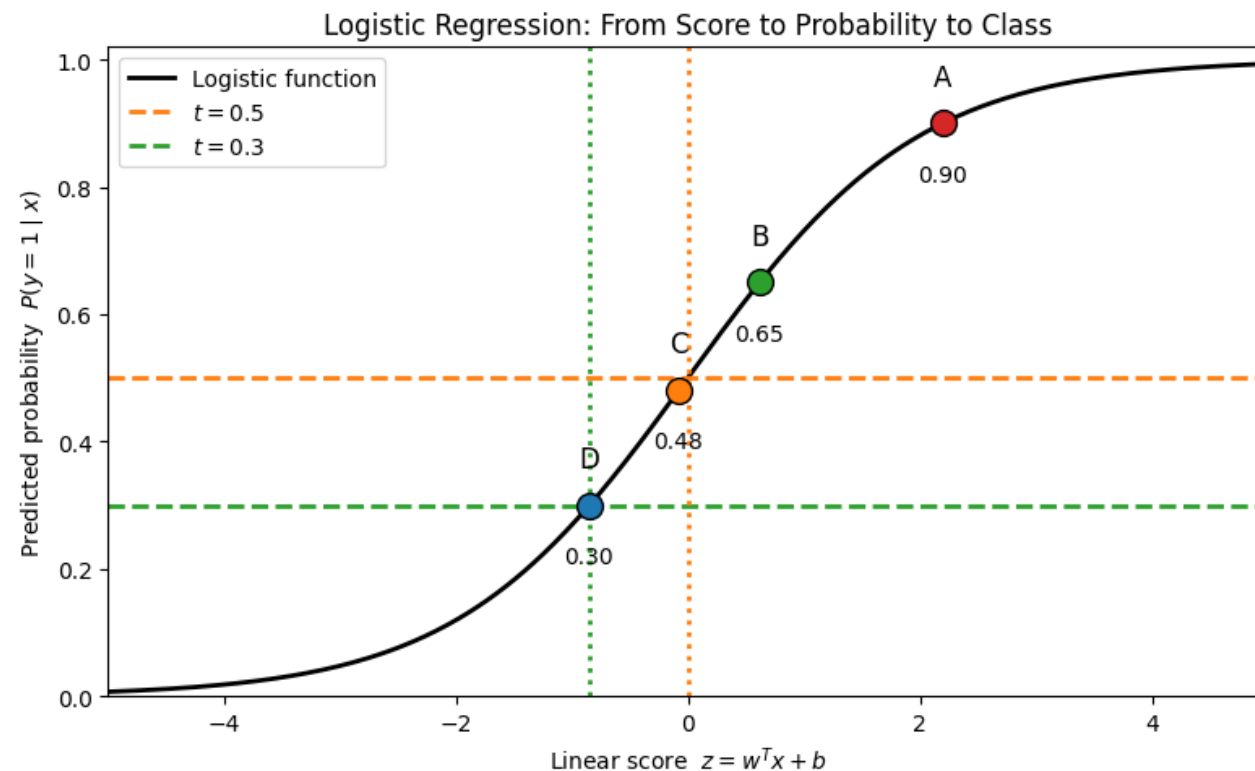
$t \downarrow \Rightarrow$ more positive predictions $\Rightarrow$ Recall $\uparrow \Rightarrow$ Precision $\downarrow$

How the Threshold Changes Predictions

$$\hat{y} = \begin{cases} 1, & P(y = 1 \mid \mathbf{x}) \geq t \\ 0, & P(y = 1 \mid \mathbf{x}) < t \end{cases}$$

- Logistic regression produces a score or probability.
- The threshold produces the class prediction.
- $t = 0.5$ is only a default, not necessarily the best threshold.

Sample	$P(y = 1)$
A	0.90
B	0.65
C	0.48
D	0.30



ROC: One Point per Threshold

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP + FN}$$

TPR = Recall or Sensitivity

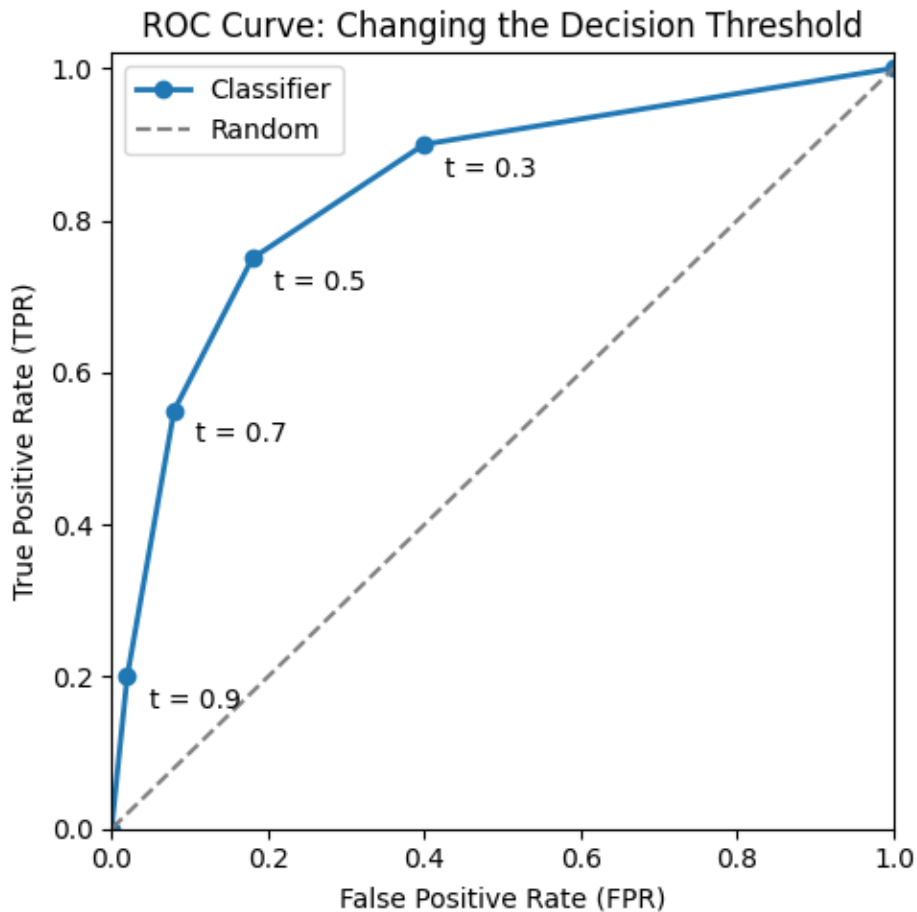
$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN}$$

$FPR = 1 - \text{Specificity}$

Threshold	TPR	FPR
0.9	0.20	0.02
0.7	0.55	0.08
0.5	0.75	0.18
0.3	0.90	0.40

- Each threshold t , one (FPR, TPR) point.
- Sweep the threshold from high to low → trace out the **ROC** curve.

The ROC Curve



- Instead of choosing one threshold immediately, the ROC curve evaluates the classifier across many thresholds.
- **Closer to upper-left is better.**
 - *We want to detect as many positives as possible, so we want a high true-positive rate. But we also want to avoid falsely labeling negatives as positive, so we want a low false-positive rate.*
- At a **very high threshold**, almost everything negative. $TPR \approx 0$, $FPR \approx 0$.
- At a **very low threshold**, almost everything positive. $TPR \approx 1$, $FPR \approx 1$.
- So the ROC curve goes approximately: $(0,0) \rightarrow (1,1)$.

RESEARCH ARTICLE

Open Access

Predicting disease risks from highly imbalanced data using random forest

Mohammed Khalilia¹, Sounak Chakraborty² and Mihail Popescu^{3*}

TPR and FPR are normalized within each class, so they do not directly depend on how many positives versus negatives you have.

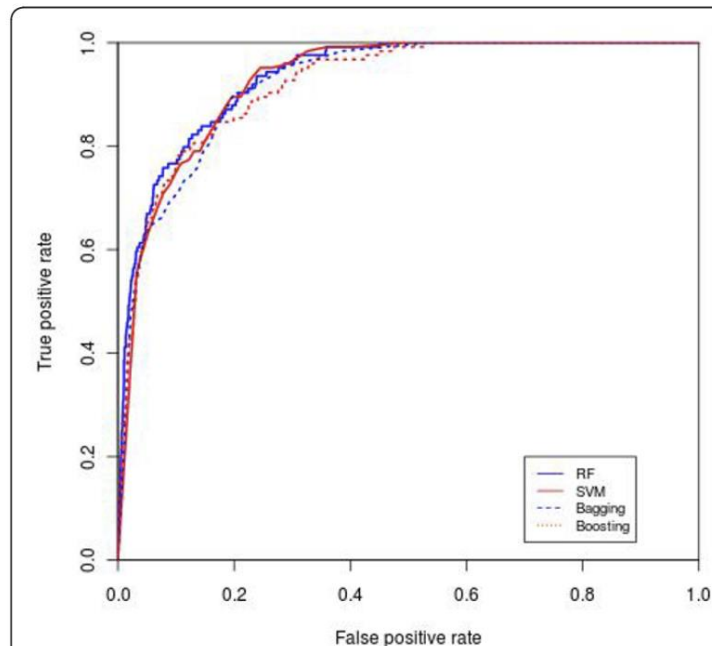


Figure 5 ROC curve for diabetes mellitus. ROC curve for diabetes mellitus comparing SVM, RF, boosting and bagging.

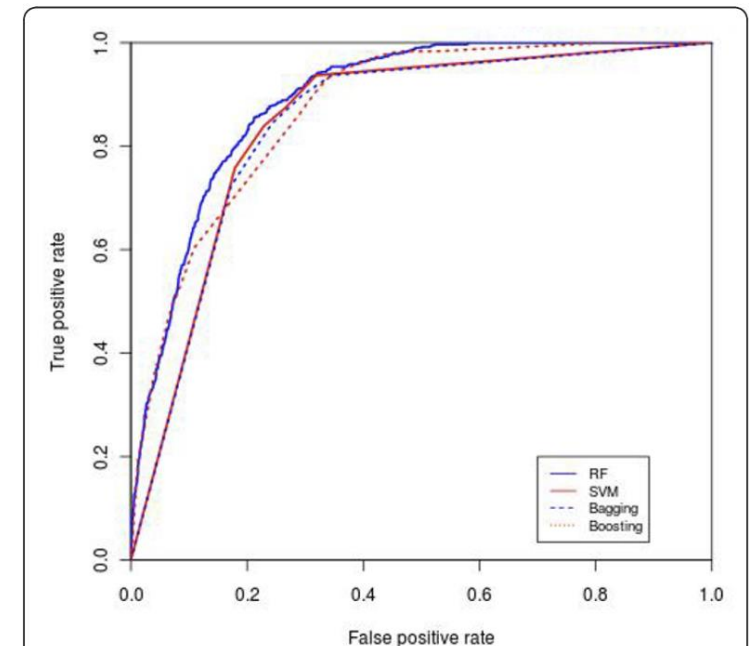
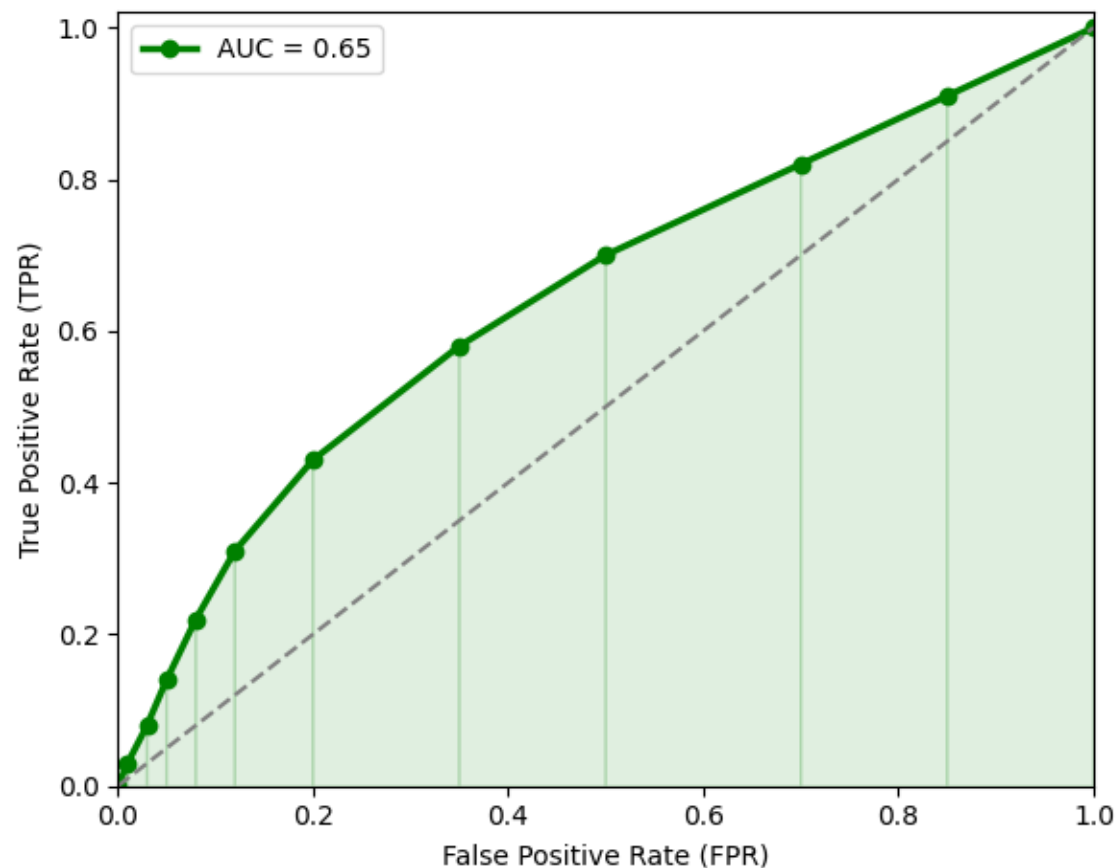


Figure 6 ROC curve for hypertension. ROC curve for hypertension comparing both SVM, RF, boosting and bagging.

ROC-AUC Measures Score Ranking



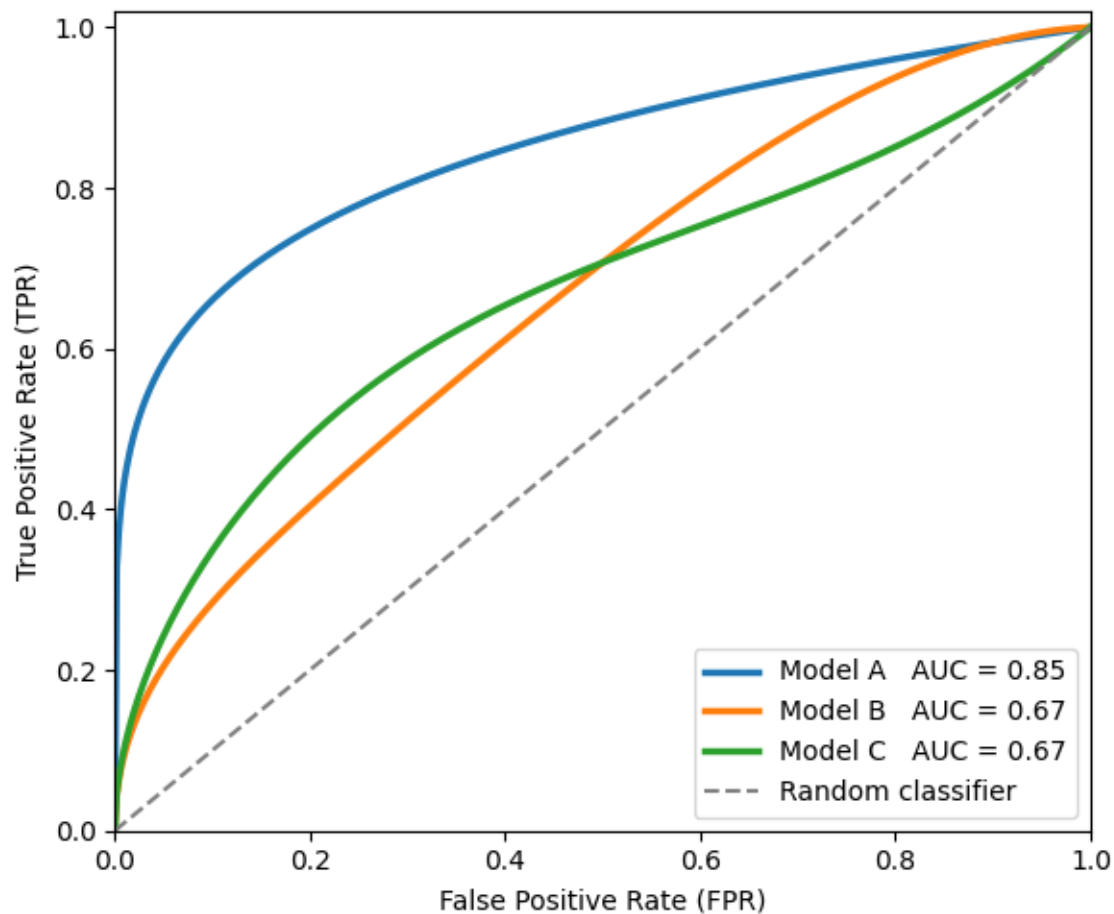
$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

Area under the curve (**AUC**) measures how well the model ranks positive samples above negative samples across all thresholds.

- $\text{AUC} = 1.0$: perfect ranking
- $\text{AUC} = 0.5$: random ranking
- $\text{AUC} < 0.5$: systematically reversed ranking

A larger overall AUC does not mean a model is better at every operating threshold.

Same AUC, Different Operating Points

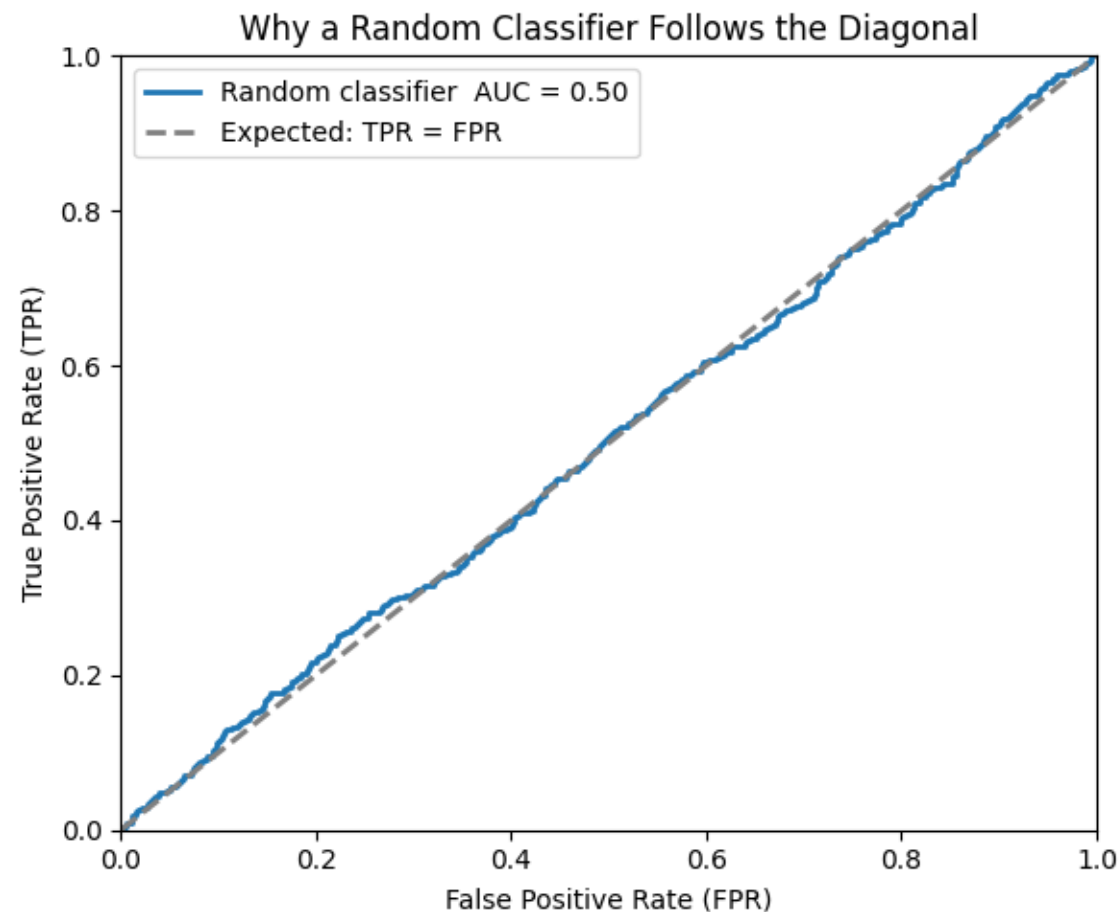


$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

Area under the curve (**AUC**) measures how well the model ranks positive samples above negative samples across all thresholds.

- Model A clearly dominates.
- Models B and C cross but have the same AUC.

Random Guessing on the ROC Curve



$$\text{ROC-AUC} = \int_0^1 \text{TPR}(\text{FPR}) d(\text{FPR})$$

Area under the curve (**AUC**) measures how well the model ranks positive samples above negative samples across all thresholds.

- A random classifier cannot distinguish positives from negatives, so it captures both at the same rate.

$$\text{True Positive Rate (TPR)} = \frac{TP}{TP + FN}$$

$$\text{False Positive Rate (FPR)} = \frac{FP}{FP + TN}$$

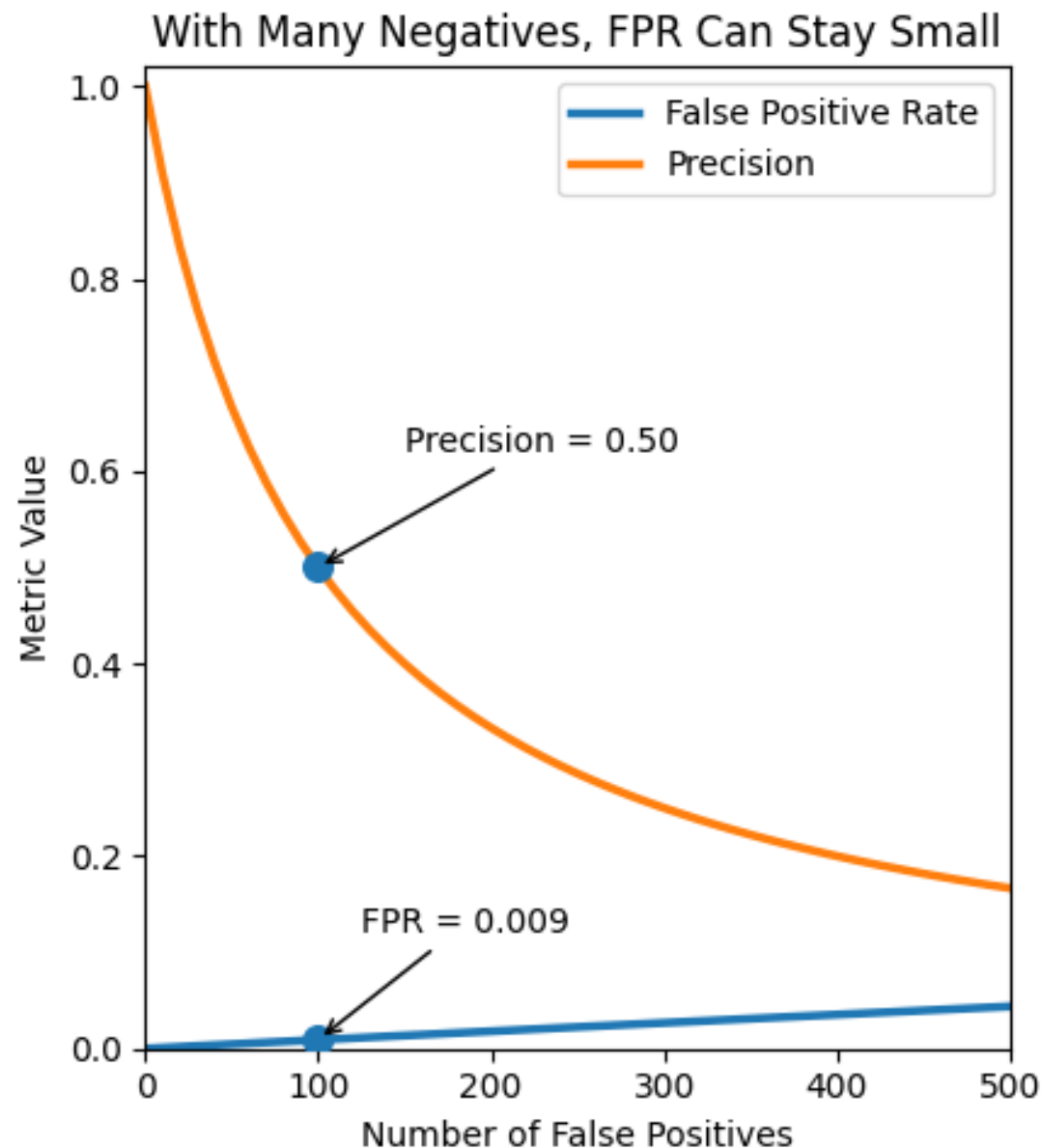
Why a Low FPR Can Still Mean Many False Alarms

$$\text{FPR} = \frac{TP}{FP + TN}$$

- When the negative class is enormous, TN is enormous too.
- So even quite a few false positives can produce a small FPR.

$$\text{Precision} = \frac{TP}{TP + FP}$$

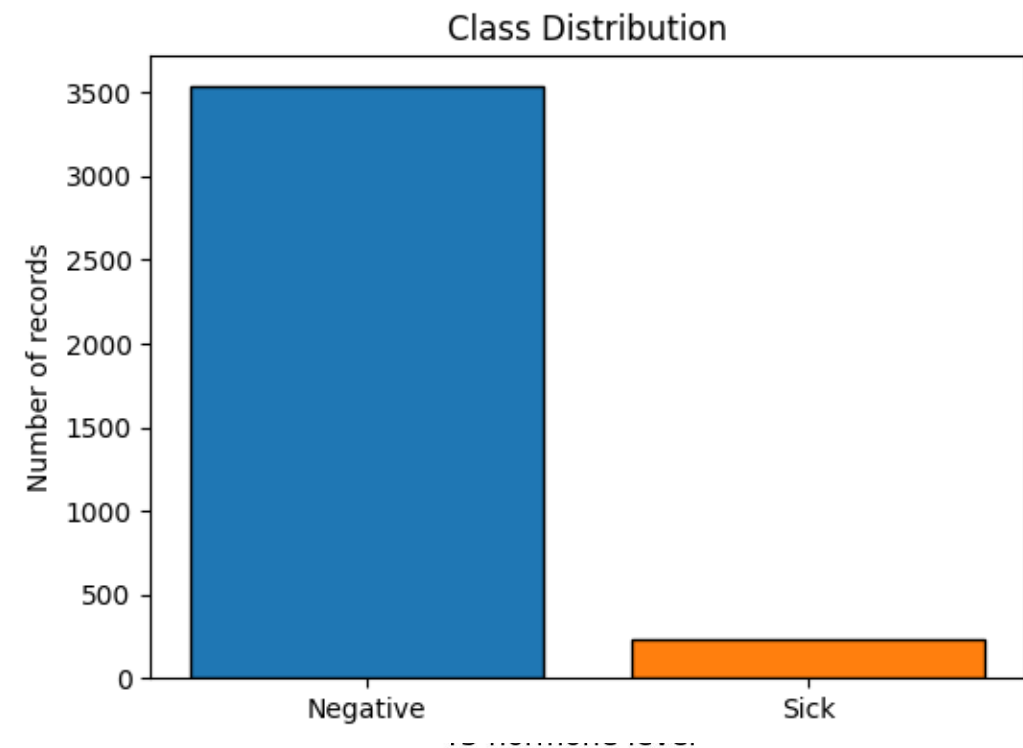
- False positives directly hurt precision.



The Thyroid “Sick” Prediction Problem

Goal: predict whether a patient record is labeled “**sick**” or “**negative**” using thyroid-related clinical history and laboratory measurements. It is a real binary classification dataset originating from thyroid disease records supplied by the Garavan Institute and Ross Quinlan.

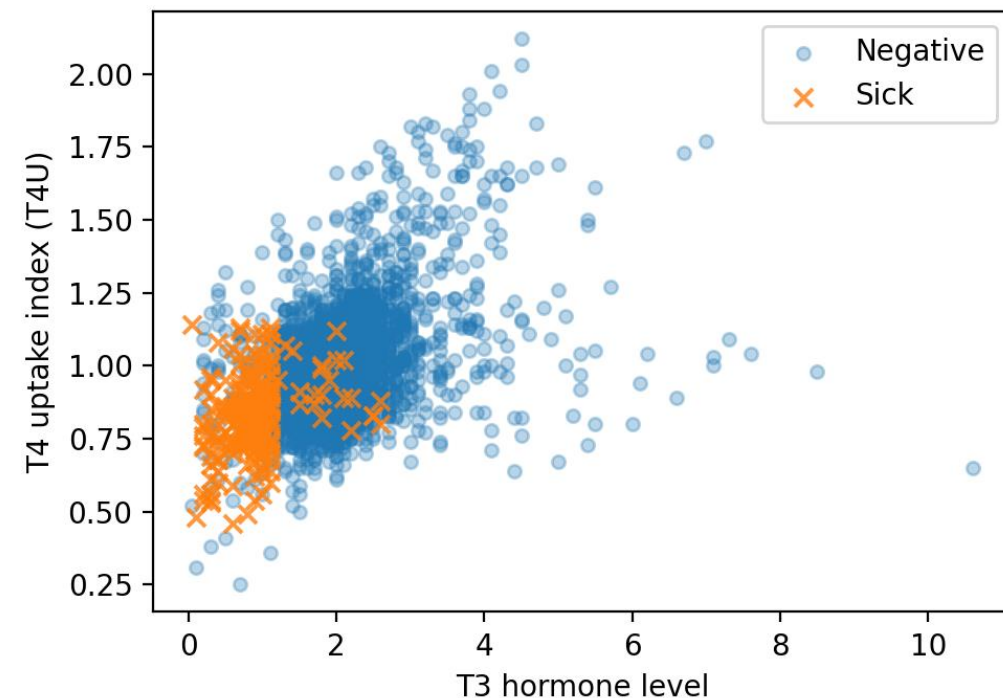
Property	Value
Samples	3,772
Features	29
Classes	2
Negative	3,541 (93.9%)
Sick	231 (6.1%)
Majority : minority	15.3 : 1
Missing values	5.4% of entries



The Thyroid “Sick” Prediction Problem

Goal: predict whether a patient record is labeled “**sick**” or “**negative**” using thyroid-related clinical history and laboratory measurements. It is a real binary classification dataset originating from thyroid disease records supplied by the Garavan Institute and Ross Quinlan.

Property	Value
Samples	3,772
Features	29
Classes	2
Negative	3,541 (93.9%)
Sick	231 (6.1%)
Majority : minority	15.3 : 1
Missing values	5.4% of entries



Thyroid Data: Load and Split

1. Load data and define positive class

```
import numpy as np
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml

data = fetch_openml(
    data_id=38,
    as_frame=True
)

X = data.data
y_raw = data.target

# sick = positive class
y = (y_raw.astype(str).str.lower() == "sick").astype(int)

print("Shape:", X.shape)
print("Negative:", np.sum(y == 0))
print("Positive:", np.sum(y == 1))
print("Positive prevalence:", y.mean())
```

```
Shape: (3772, 29)
Negative: 3541
Positive: 231
Positive prevalence: 0.0612407211028632
```

2. Train/test split

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer, make_column_selector
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.25,
    stratify=y,
    random_state=26
)
```

Thyroid Model: Preprocessing and Evaluation Code

3. Cleaning Pipeline

```
# Numerical features
numeric = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

# Categorical features
categorical = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

preprocess = ColumnTransformer([
    ("num", numeric,
     make_column_selector(dtype_include=np.number)),

    ("cat", categorical,
     make_column_selector(dtype_exclude=np.number))
])

model = Pipeline([
    ("preprocess", preprocess),
    ("logistic", LogisticRegression(
        max_iter=2000,
        random_state=26
    ))
])

model.fit(X_train, y_train)
```

4. ROC and Precision-Recall curves

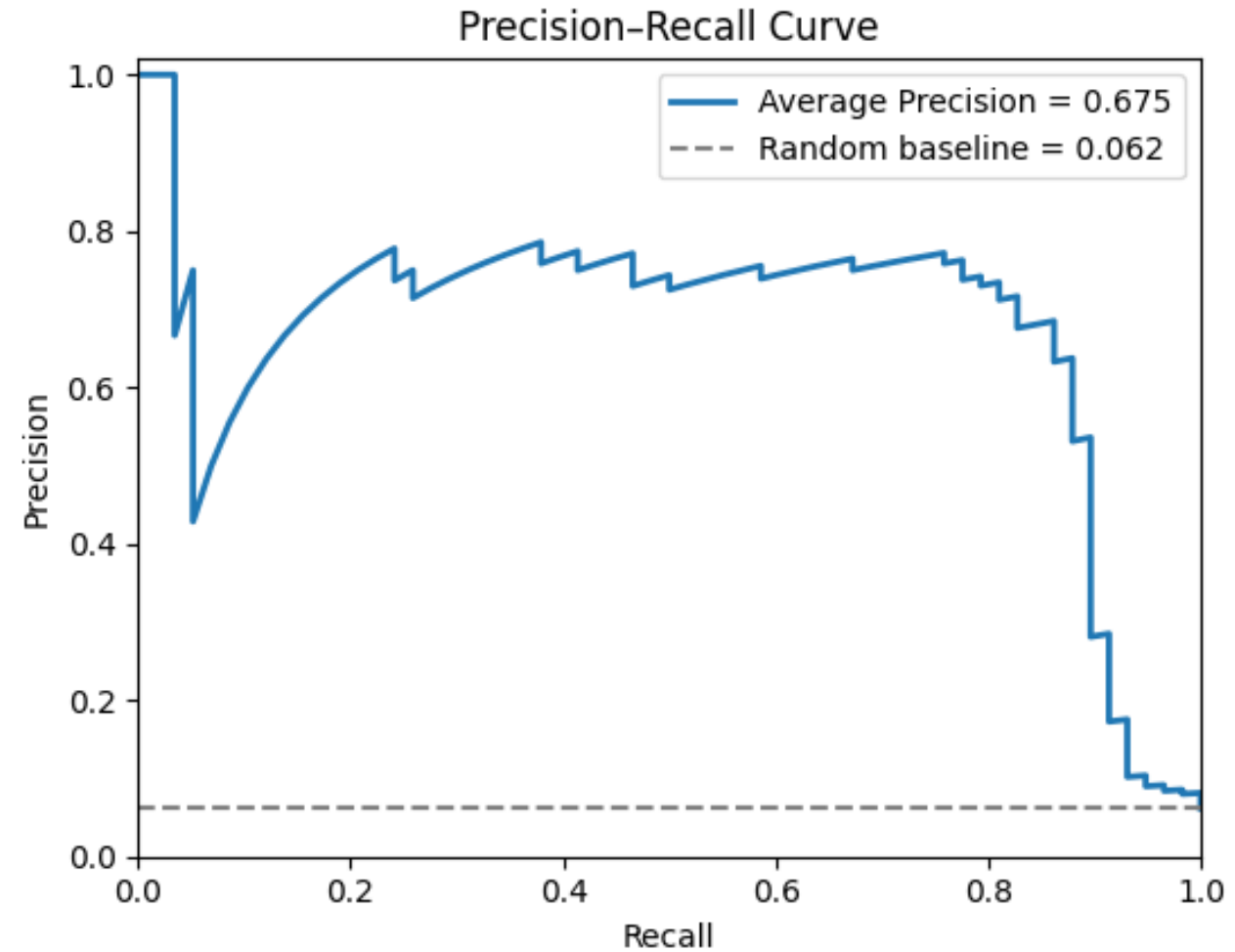
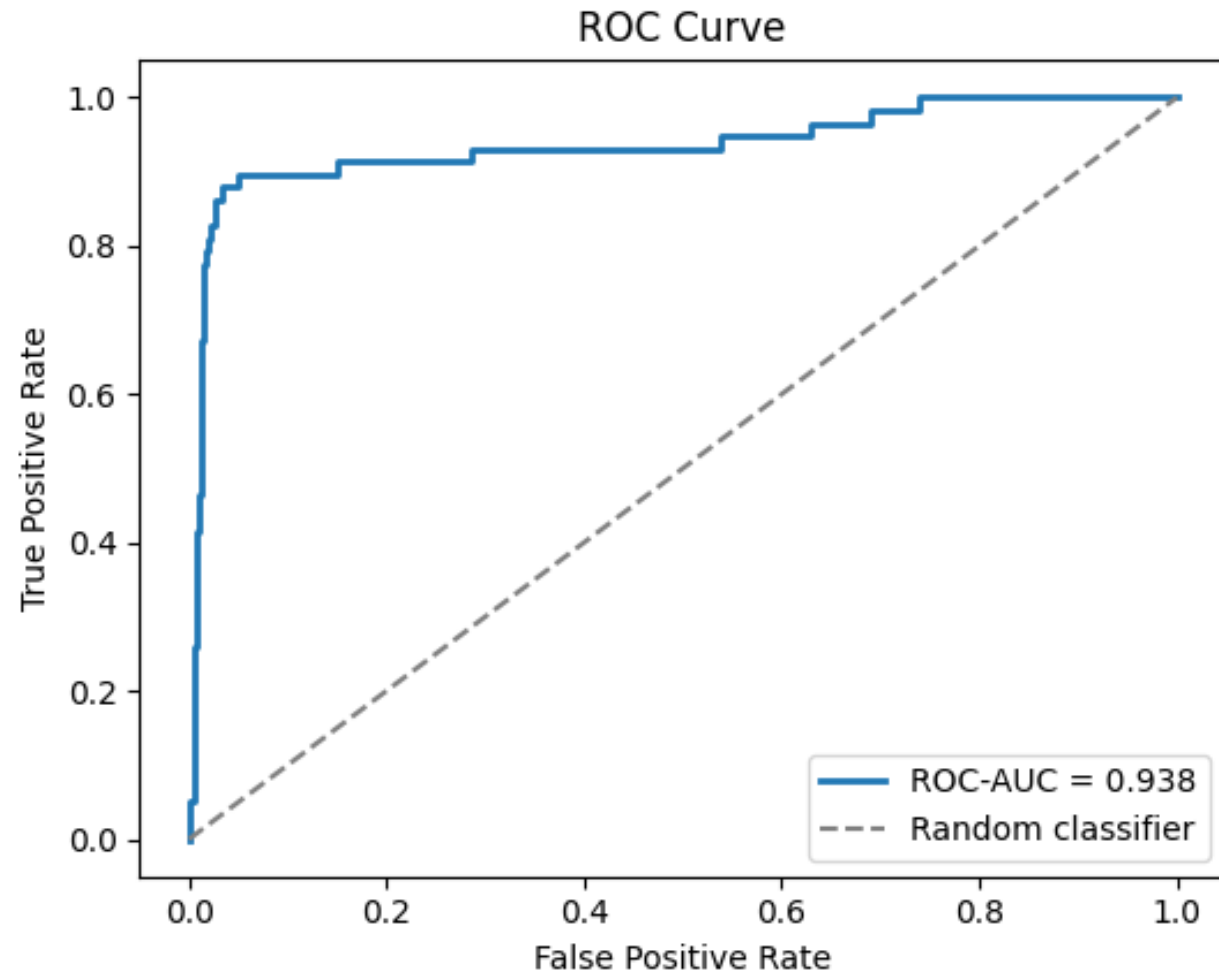
```
from sklearn.metrics import (
    roc_curve,
    roc_auc_score,
    precision_recall_curve,
    average_precision_score
)

# Predicted probability of "sick"
p = model.predict_proba(X_test)[:, 1]

# ROC
fpr, tpr, _ = roc_curve(y_test, p)
roc_auc = roc_auc_score(y_test, p)

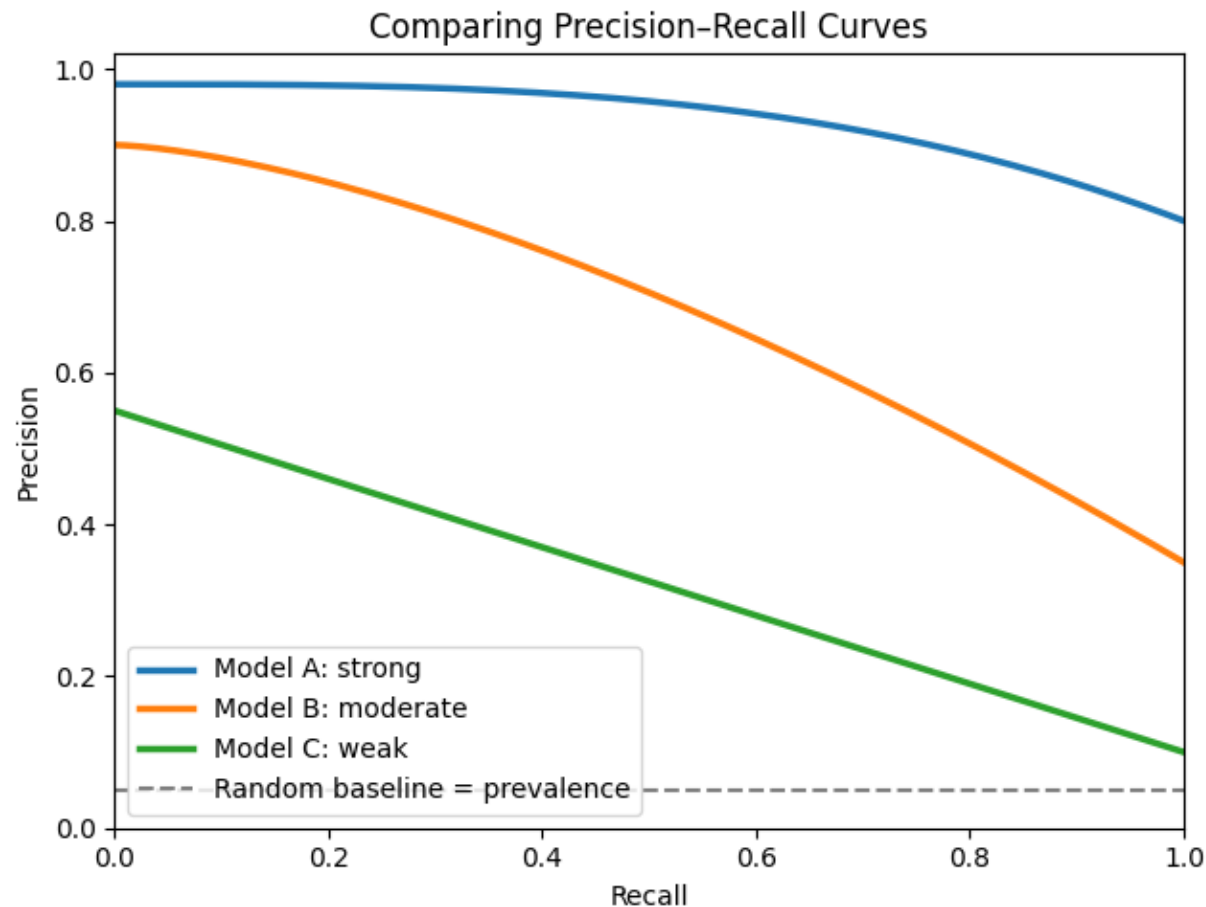
# Precision-Recall
precision, recall, _ = precision_recall_curve(y_test, p)
ap = average_precision_score(y_test, p)
```

Thyroid Model: ROC and Precision–Recall Curves



- For PR curve, x-axis recall means: of all real positive cases, how many did we find?
- y-axis precision means: of everything we predicted as positive, how many were actually positive?

Comparing Precision-Recall Curves



- Model A: maintains high precision even as recall increases → good.
- Model B: precision falls substantially as we try to capture more positives.
- Model C: to achieve high recall, we must accept many false positives.

Good PR curve = high precision maintained at high recall

What Average Precision Summarizes

Average Precision (AP) summarizes precision across different recall levels.

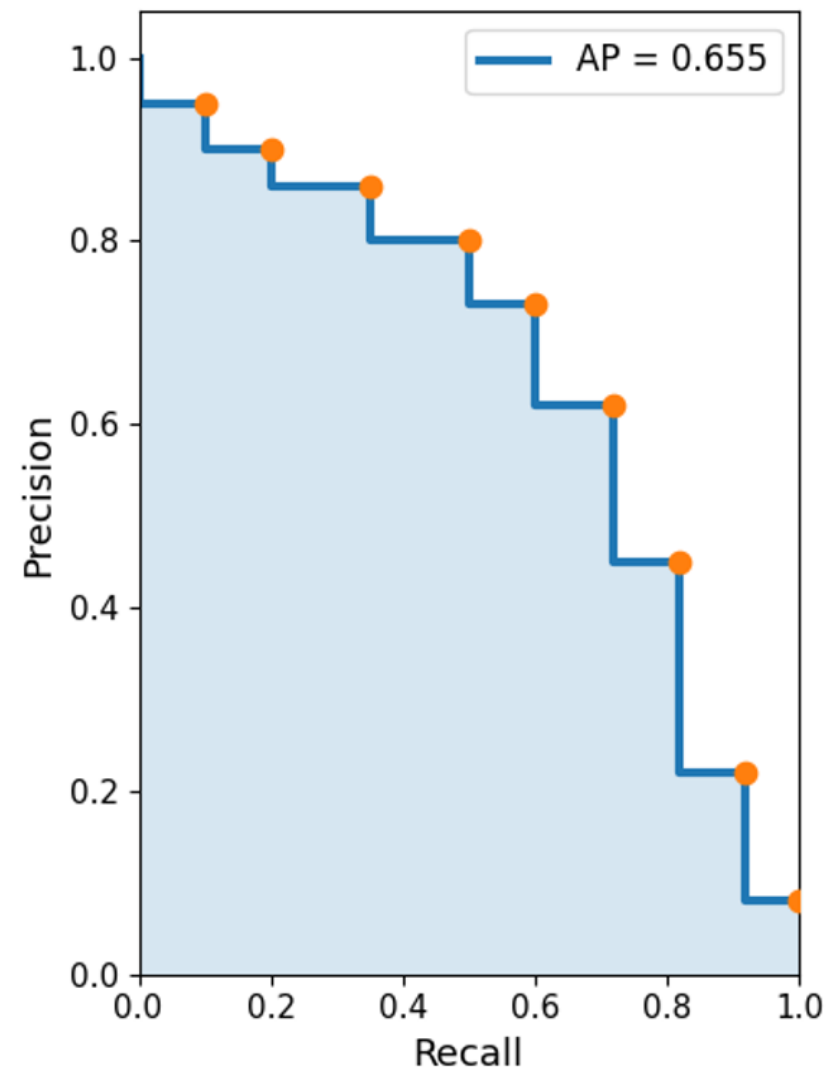
$$AP = \sum_n (R_n - R_{n-1}) P_n$$

where:

- P_n = precision at threshold n
- R_n = recall at threshold n

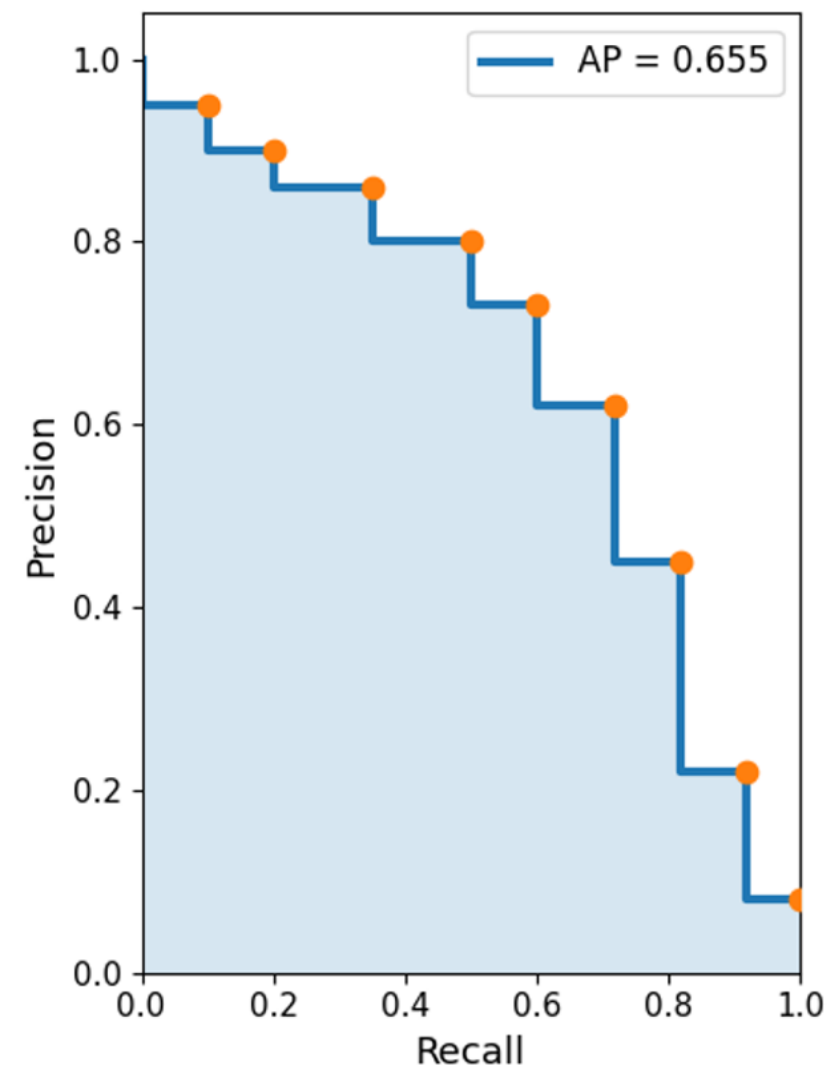
Conceptually, AP is high when the model maintains high precision while recall increases.

Average Precision as Summed Area



Average Precision as Stepwise Area

Average Precision as Summed Area



- For a strong classifier: Precision stays high as recall $\rightarrow 1$. So, AP $\rightarrow 1$.
- For a weak classifier, precision falls rapidly as recall increases: AP $\rightarrow$ class prevalence.
- For example, if only 5% of samples are positive. Random baseline AP ≈ 0.05 .
- So unlike ROC-AUC: Random ROC-AUC = 0.5.
- For RP: Random AP $\approx N+ / N$.

ROC-AUC vs Average Precision

	ROC-AUC	Average Precision (AP)
Based on	ROC curve	Precision–Recall curve
Measures	Ranking of positives vs. negatives	Precision maintained as recall increases
Random baseline	0.5	Positive-class prevalence
Best value	1.0	1.0
Sensitive to class imbalance?	Less sensitive	More sensitive
Best use	General ranking performance	Rare positive / imbalanced classification
Main limitation	Can look strong despite many false positives	Depends strongly on prevalence

Class-Weighted Logistic Regression

Unweighted logistic regression

$$L = - \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Weighted logistic regression

$$L = - \sum_i w_{y_i} [y_i \log p_i + (1 - y_i) \log(1 - p_i)]$$

Each class receives a different weight.

class_weight : dict or 'balanced', default=None

Weights associated with classes in the form `{class_label: weight}`. If not given, all classes are supposed to have weight one.

The "balanced" mode uses the values of y to automatically adjust weights inversely proportional to class frequencies in the input data as `n_samples / (n_classes * np.bincount(y))`.

For class_weight="balanced"

$$w_k = \frac{N}{K N_k}$$

$$N = 1000, N_0 = 980, N_1 = 20$$

$$w_0 = \frac{1000}{2 \cdot 980} \approx 0.51$$

$$w_1 = \frac{1000}{2 \cdot 20} = 25$$

Fitting Ordinary and Weighted Logistic Regression

```
from sklearn.base import clone
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

# Standard logistic regression
model_normal = Pipeline([
    ("preprocess", clone(preprocess)),
    ("lr", LogisticRegression(
        max_iter=2000,
        random_state=26
    ))
])

# Class-weighted logistic regression
model_weighted = Pipeline([
    ("preprocess", clone(preprocess)),
    ("lr", LogisticRegression(
        class_weight="balanced",
        max_iter=2000,
        random_state=26
    ))
])

model_normal.fit(X_train, y_train)
model_weighted.fit(X_train, y_train)
```

Clone the previous preprocess steps

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

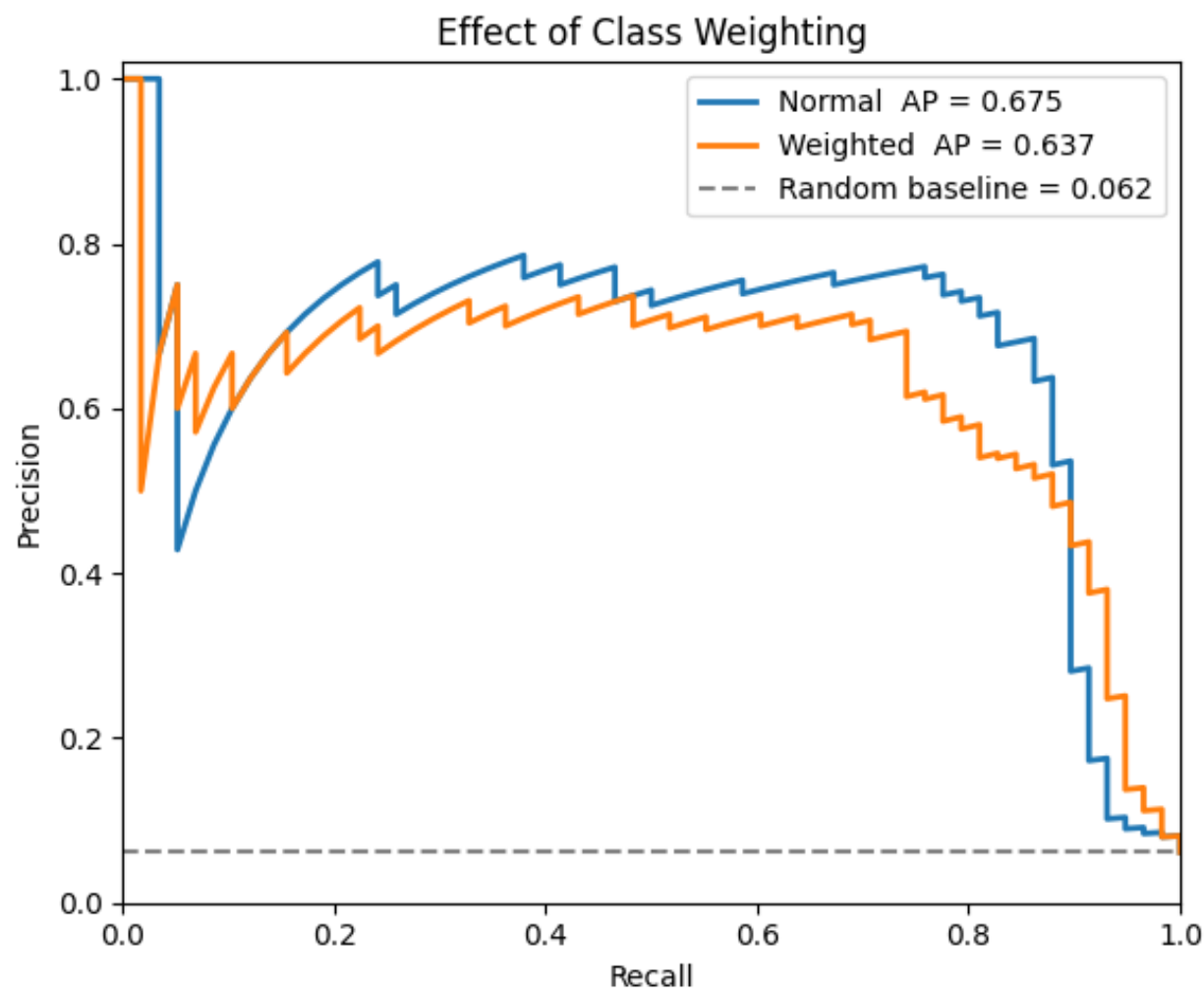
for name, model in [
    ("Normal", model_normal),
    ("Weighted", model_weighted)
]:
    y_pred = model.predict(X_test)

    print(name)
    print("Accuracy :", round(accuracy_score(y_test, y_pred), 3))
    print("Precision:", round(precision_score(y_test, y_pred), 3))
    print("Recall    :", round(recall_score(y_test, y_pred), 3))
    print("F1       :", round(f1_score(y_test, y_pred), 3))
    print()
```

Normal
Accuracy : 0.962
Precision: 0.75
Recall : 0.569
F1 : 0.647

Weighted
Accuracy : 0.907
Precision: 0.39
Recall : 0.914
F1 : 0.546

Weighting Trades Misses for False Alarms



Normal Logistic Regression

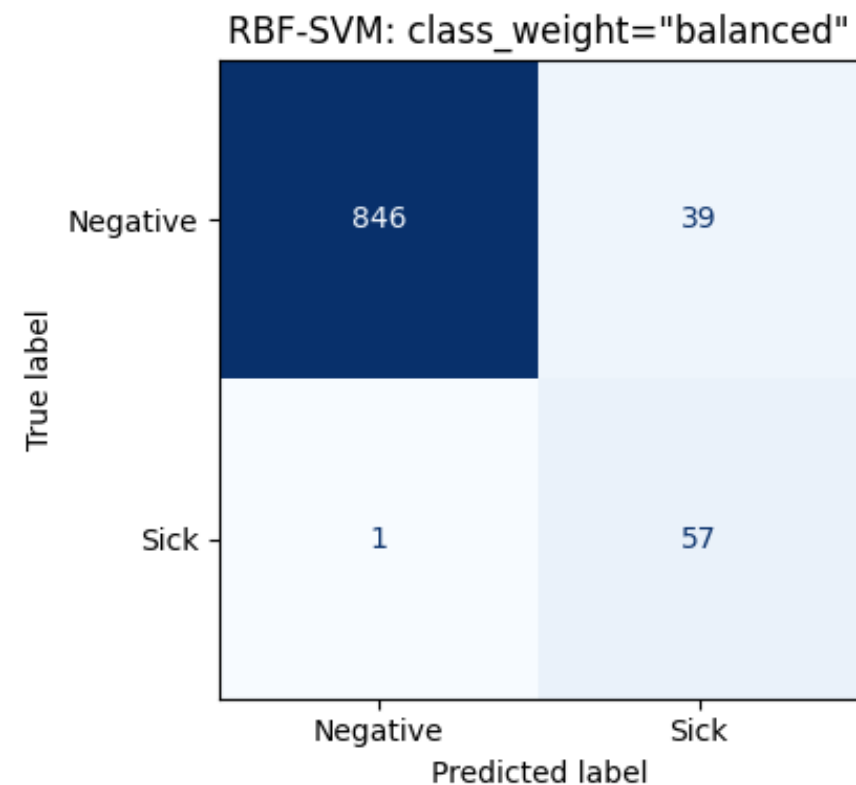
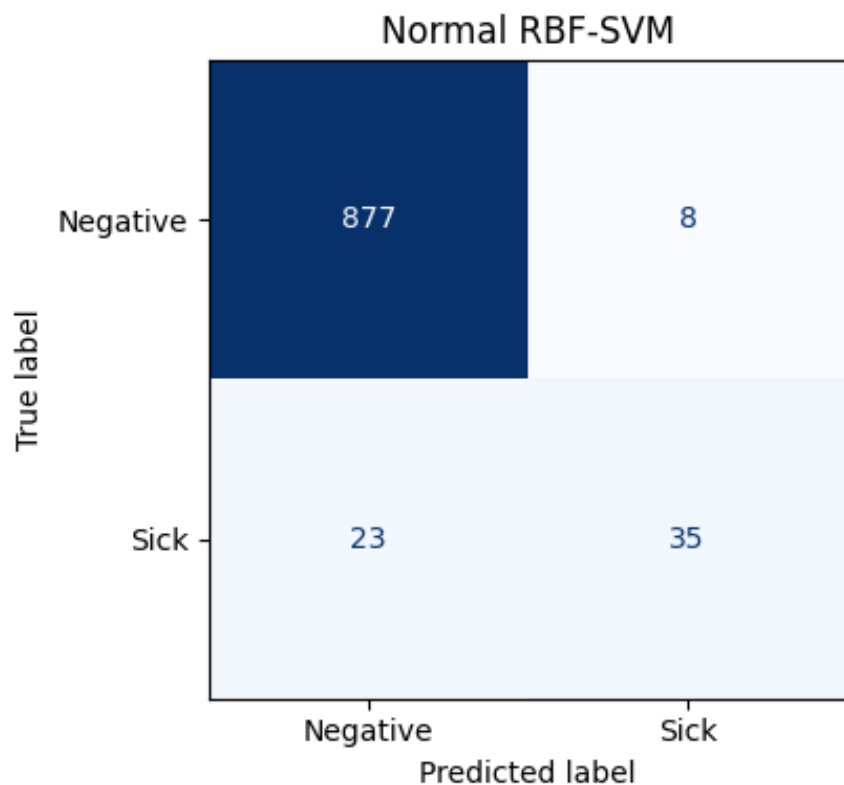
True label	Predicted label	
	Negative	Sick
Negative	874	11
Sick	25	33

Class Weight = "balanced"

True label	Predicted label	
	Negative	Sick
Negative	802	83
Sick	5	53

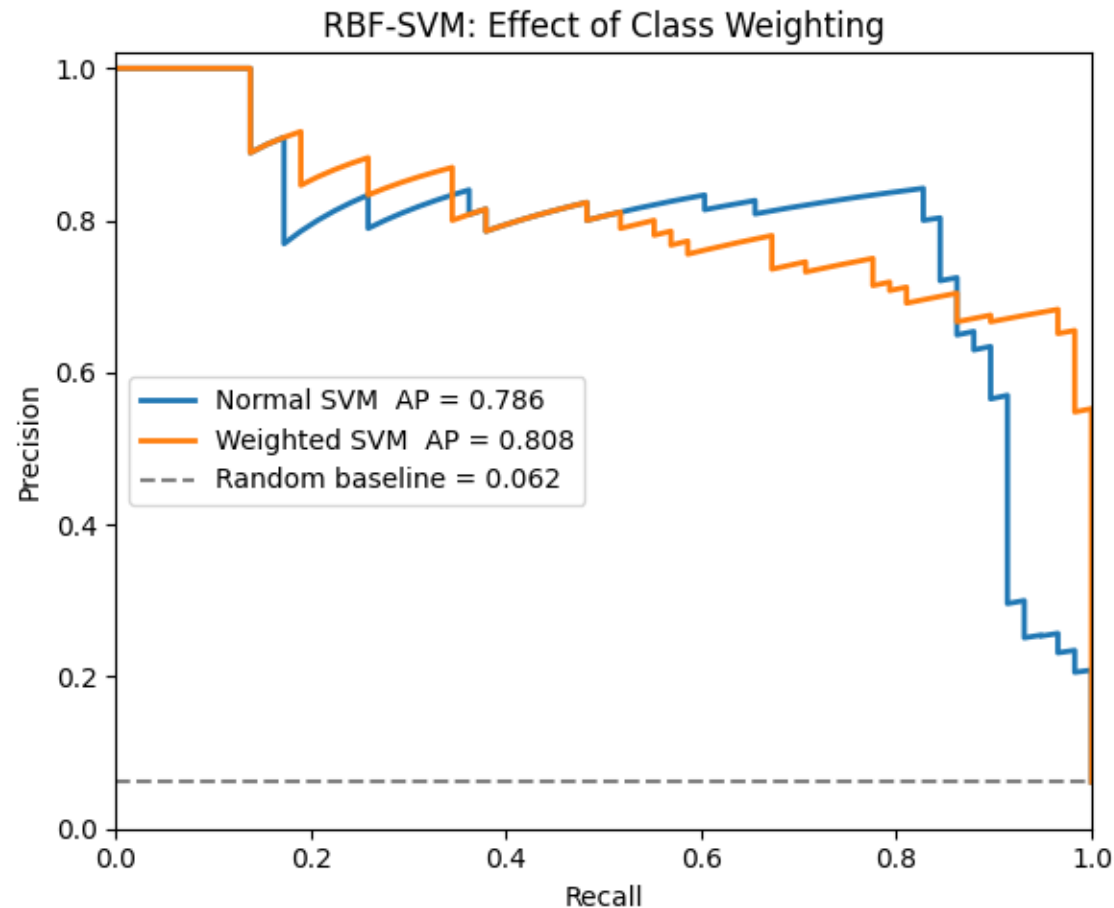
Class Weighting in an RBF SVM

```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale',  
coef0=0.0, shrinking=True, probability='deprecated', tol=0.001,  
cache_size=200, class_weight=None, verbose=False, max_iter=-1,  
decision_function_shape='ovr', break_ties=False, random_state=None)
```



RBF SVM: Precision–Recall Results

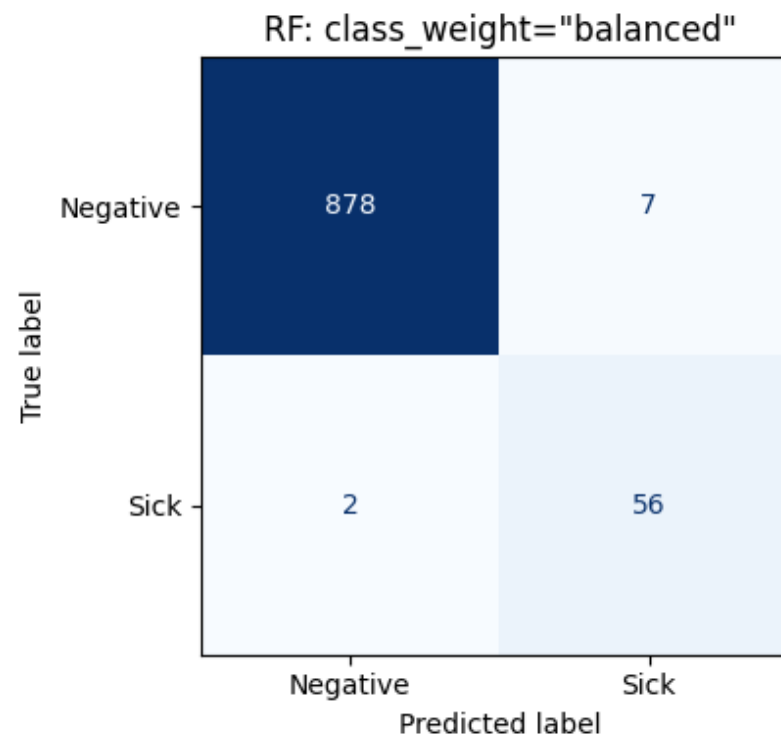
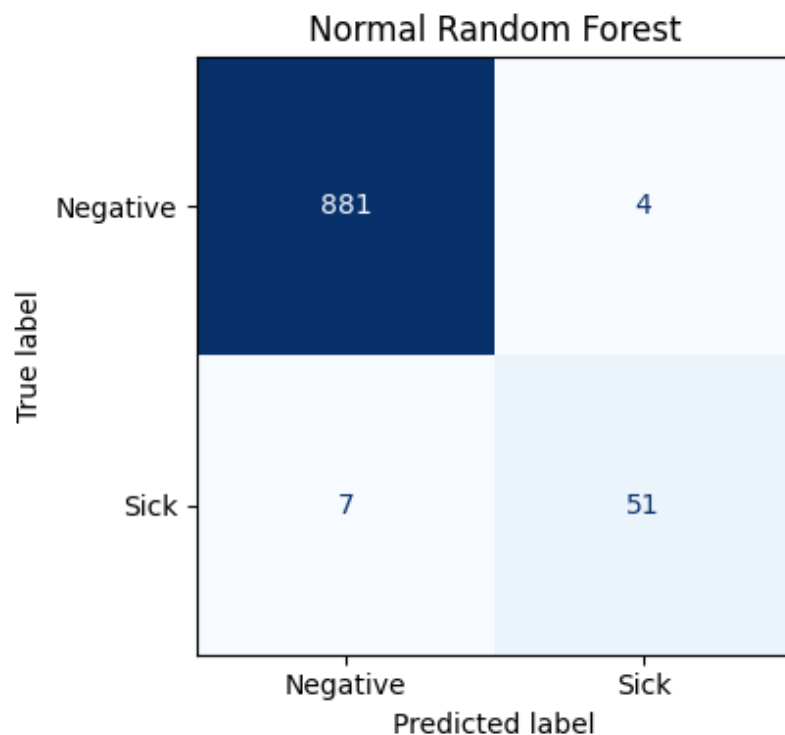
```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale',  
coef0=0.0, shrinking=True, probability='deprecated', tol=0.001,  
cache_size=200, class_weight=None, verbose=False, max_iter=-1,  
decision_function_shape='ovr', break_ties=False, random_state=None)
```



Class Weighting in a Random Forest

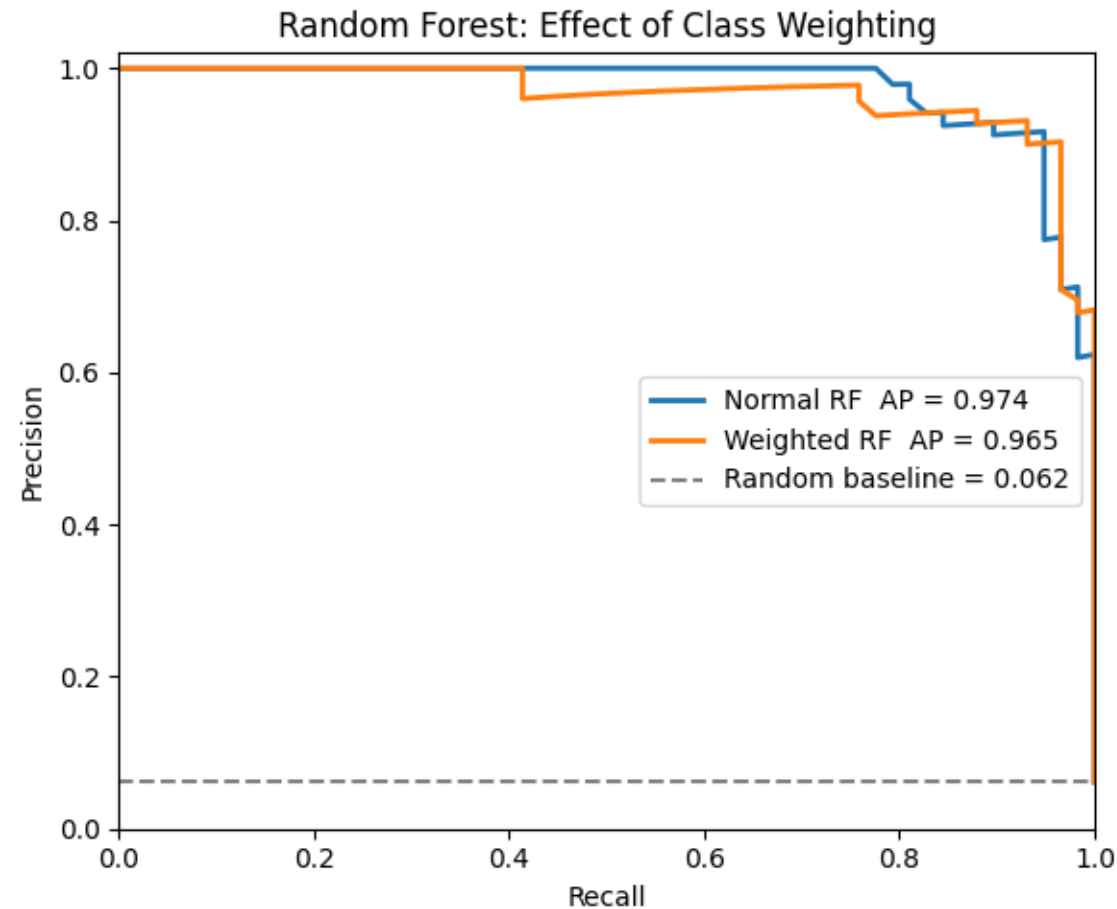
```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *,  
criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1,  
min_weight_fraction_leaf=0.0, max_features='sqrt', max_leaf_nodes=None,  
min_impurity_decrease=0.0, bootstrap=True, oob_score=False, n_jobs=None,  
random_state=None, verbose=0, warm_start=False, class_weight=None,  
ccp_alpha=0.0, max_samples=None, monotonic_cst=None)
```

[\[sou\]](#)



Random Forest: Precision–Recall Results

```
class sklearn.svm.SVC(*, C=1.0, kernel='rbf', degree=3, gamma='scale',  
coef0=0.0, shrinking=True, probability='deprecated', tol=0.001,  
cache_size=200, class_weight=None, verbose=False, max_iter=-1,  
decision_function_shape='ovr', break_ties=False, random_state=None)
```



A fitted model assigns each sample a score. You lower its decision threshold. Which quantities can change?

- A. Precision, recall, and the confusion matrix
- B. ROC-AUC and AP
- C. The learned model coefficients
- D. Positive-class prevalence

Answer: A. The class labels change, so precision, recall, and the confusion matrix may change. The scores and their ranking do not, so ROC-AUC and AP remain fixed.

You compare six models and select the one with the highest test AP. What is the problem?

- A. AP cannot evaluate imbalanced data.
- B. The test set has become part of model selection.
- C. All six models need the same threshold.
- D. The class proportions must be 50:50.

Answer: B. Repeated comparison uses the test labels to select a model. Select with validation data, then evaluate the fixed choice on the test set once.

Interactive Python Time

`16_imbalanced_classification.ipynb`